ASSIGNMENT – 14.3

Name: B.Sri Laxmi Gayathri

H. No: 2303A52033

Batch: 38

Task 1: AI-Assisted Personal Portfolio Website

Scenario

A student wants to showcase academic projects, technical skills, and contact information through a personal portfolio website. AI tools are used to speed up development while maintaining quality and customization.

Tasks

1. Use GitHub Copilot to generate a semantic HTML structure with sections:

o Home

o About

o Projects

o Contact

2. Ask Copilot to suggest responsive CSS using Flexbox or Grid.

3. Customize AI-generated CSS to add:

o Hover effects on project cards

o Smooth scrolling navigation

Expected Outcome

• Clean and well-structured portfolio website

• Responsive layout for desktop and mobile screens

• Interactive project cards

PROMPT:

```
<!-- Generate a responsive personal portfolio website using a SINGLE HTML file.
Requirements:
1. Use semantic HTML5 structure.
2. Include the following sections:
   * Home
   * About
   * Projects
   * Contact
3. Navigation:
   * Add a navigation bar with links to each section.
   * Implement smooth scrolling when clicking navigation links.
4. Projects Section:
   * Create at least 3 project cards.
   * Each card should contain a project title, short description, and a button.
   * Add hover effects on the project cards.
5. Contact Section:
   * Include a simple contact form with Name, Email, and Message fields.
6. Styling:
   * Write CSS inside a `<style>` tag in the same HTML file.
   * Use Flexbox or CSS Grid for layout.
   * Make the website responsive for both desktop and mobile screens.
   * Add smooth transitions and modern styling.
7. JavaScript:
   * Write JavaScript inside a `<script>` tag at the bottom of the HTML file.
   * Implement smooth scrolling navigation.
```

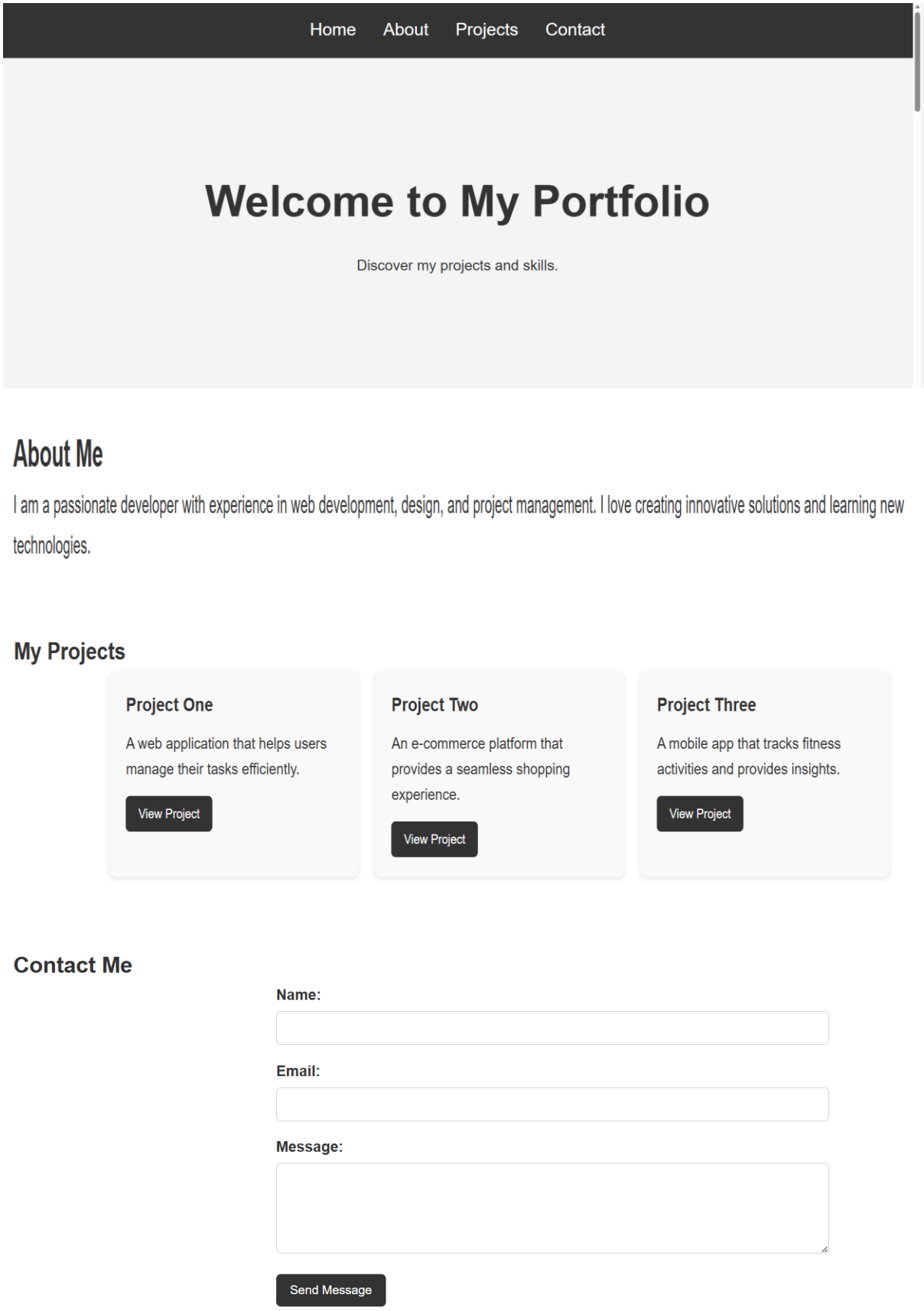
CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
    <title>Personal Portfolio</title>
    <style>
        /* Reset some default styles */
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
            color: #333;
        }
        /* Navigation Bar */
        nav {
            background-color: #333;
            color: #fff;
            padding: 1rem;
            position: fixed;
            width: 100%;
            top: 0;
            z-index: 1000;
```

```html
</head>
<body>
    <nav>
        <ul>
            <li><a href="#home">Home</a></li>
            <li><a href="#about">About</a></li>
            <li><a href="#projects">Projects</a></li>
            <li><a href="#contact">Contact</a></li>
        </ul>
    </nav>
    <section id="home">
        <h1>Welcome to My Portfolio</h1>
        <p>Discover my projects and skills.</p>
    </section>

    <section id="about">
        <h2>About Me</h2>
        <p>I am a passionate developer with experience in web development, design, and project management
    </section>
    <section id="projects">
        <h2>My Projects</h2>
        <div class="projects-container">
            <div class="project-card">
                <h3>Project One</h3>
```

```html
<section id="projects">
    <h2>My Projects</h2>
    <div class="projects-container">
        <div class="project-card">
            <h3>Project One</h3>
            <p>A web application that helps users manage their tasks efficiently.</p>
            <button>View Project</button>
        </div>
        <div class="project-card">
            <h3>Project Two</h3>
            <p>An e-commerce platform that provides a seamless shopping experience.</p>
            <button>View Project</button>
        </div>
        <div class="project-card">
            <h3>Project Three</h3>
            <p>A mobile app that tracks fitness activities and provides insights.</p>
            <button>View Project</button>
        </div>
    </div>
</section>
<section id="contact">
    <h2>Contact Me</h2>
    <form class="contact-form">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>
```

OUTPUT:


Home
About
Projects
Contact
Welcome to My Portfolio
Discover my projects and skills.
About Me
I am a passionate developer with experience in web development, design, and project management. I love creating innovative solutions and learning new technologies.
My Projects
Project One
A web application that helps users manage their tasks efficiently.
View Project
Project Two
An e-commerce platform that provides a seamless shopping experience.
View Project
Project Three
A mobile app that tracks fitness activities and provides insights.
View Project
Contact Me
Name:
Email:
Message:
Send Message

Task 2: AI-Generated Restaurant Landing Page

Scenario

A local restaurant requires a simple yet attractive landing page to display its offerings. The developer uses AI assistance to rapidly generate UI components.

Tasks

1. Use Copilot to create a navigation bar with links:

o Home

o Menu

o Gallery

o Contact

2. Generate a menu section styled with CSS cards.

3. Implement a JavaScript-based image slider for the gallery using Copilot suggestions.

4. Customize animations or transitions for better user experience.

Expected Outcome

• Fully functional restaurant landing page

• Responsive navigation bar

• Working image slider with smooth transitions

PROMPT:

Generate a responsive Restaurant Landing Page using a single HTML file with HTML, CSS, and JavaScript.

Requirements:

1. HTML Structure:

- Use semantic HTML5 elements.

- Create the following sections:

  • Home

  • Menu

  • Gallery

  • Contact

2. Navigation Bar:

- Create a responsive navigation bar at the top of the page.

- Include links for Home, Menu, Gallery, and Contact.

- Implement smooth scrolling when clicking the navigation links.

3. Menu Section:

- Design a menu section displaying at least 4 food items.
- Each item should be shown in a CSS card layout.
- Each card should include:
  - Food image
  - Food name
  - Short description
  - Price
- Add hover effects to the cards.

4. Gallery Section:

- Implement an image slider using JavaScript.
- Display multiple restaurant or food images.
- Include previous and next buttons for navigation.
- Add smooth transition effects between images.

5. Contact Section:

- Add a simple contact form with fields:
  - Name
  - Email
  - Message
- Include a submit button.

6. Styling:

- Write CSS inside a <style> tag.
- Use Flexbox or CSS Grid for layout.
- Add modern design with animations and smooth transitions.
- Ensure the website is fully responsive for mobile and desktop screens.

7. JavaScript:

- Write JavaScript inside a <script> tag at the bottom of the HTML file.
- Implement the image slider functionality.
- Add smooth scrolling for navigation links.

CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Restaurant Landing Page</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: 'Arial', sans-serif;
            line-height: 1.6;
            color: #333;
            scroll-behavior: smooth;
        }
        header {
            position: fixed;
            top: 0;
            width: 100%;
            background: rgba(0, 0, 0, 0.8);
            z-index: 1000;
        }
```

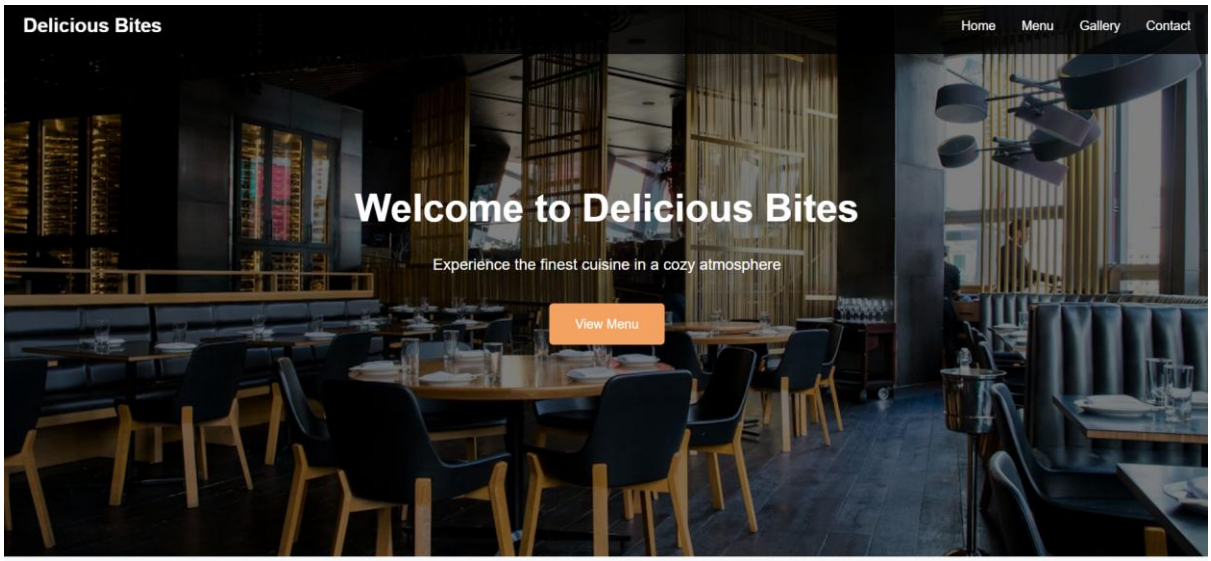
```html
    </style>
</head>
<body>
    <header>
        <nav>
            <div class="logo">Delicious Bites</div>
            <ul class="nav-links">
                <li><a href="#home">Home</a></li>
                <li><a href="#menu">Menu</a></li>
                <li><a href="#gallery">Gallery</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
            <div class="hamburger">
                <div></div>
                <div></div>
                <div></div>
            </div>
        </nav>
    </header>

    <section id="home">
        <h1>Welcome to Delicious Bites</h1>
        <p>Experience the finest cuisine in a cozy atmosphere</p>
        <a href="#menu" class="btn">View Menu</a>
    </section>

    <section id="menu">
        <h2>Our Menu</h2>
        <div class="menu-grid">
```

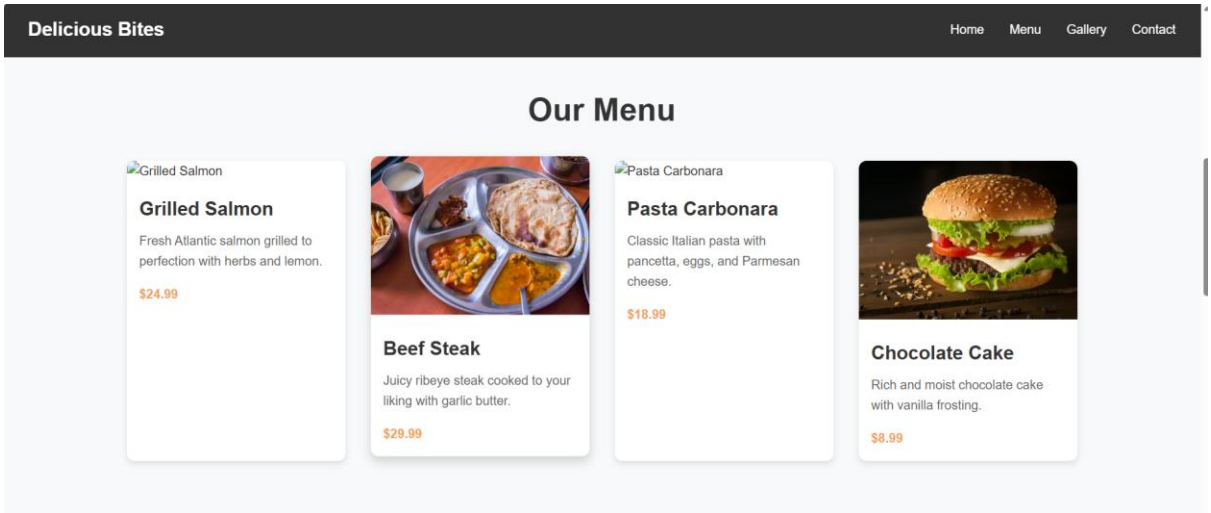
```html
div class="menu-card">
    <img src="https://images.unsplash.com/photo-1546833999-b9f581a1996d?ixlib=rb-4.0.3&auto=format&fit=crop&w=500
    <div class="menu-card-content">
        <h3>Beef Steak</h3>
        <p>Juicy ribeye steak cooked to your liking with garlic butter.</p>
        <div class="price">$29.99</div>
    </div>
/div>
div class="menu-card">
    <img src="https://images.unsplash.com/photo-1551782450-17144efb5723?ixlib=rb-4.0.3&auto=format&fit=crop&w=500
    <div class="menu-card-content">
        <h3>Pasta Carbonara</h3>
        <p>Classic Italian pasta with pancetta, eggs, and Parmesan cheese.</p>
        <div class="price">$18.99</div>
    </div>
/div>
div class="menu-card">
    <img src="https://images.unsplash.com/photo-1571091718767-18b5b1457add?ixlib=rb-4.0.3&auto=format&fit=crop&w=
    <div class="menu-card-content">
        <h3>Chocolate Cake</h3>
```

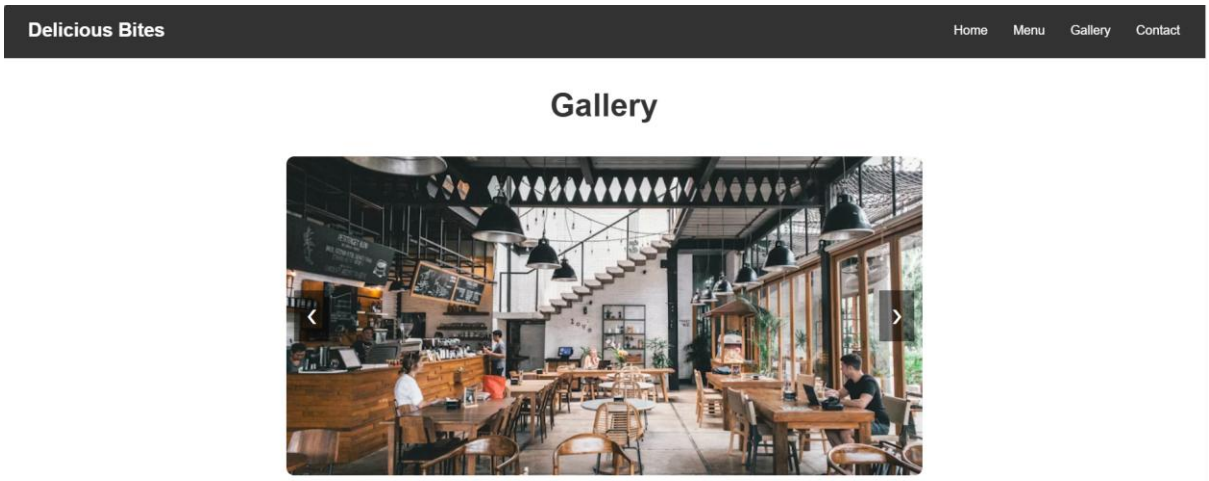
```html
<script>
    // Smooth scrolling for navigation links
    document.querySelectorAll('a[href^="#"]').forEach(anchor => {
        anchor.addEventListener('click', function (e) {
            e.preventDefault();
            document.querySelector(this.getAttribute('href')).scrollIntoView({
                behavior: 'smooth'
            });
        });
    });

    // Hamburger menu toggle
    const hamburger = document.querySelector('.hamburger');
    const navLinks = document.querySelector('.nav-links');

    hamburger.addEventListener('click', () => {
        navLinks.classList.toggle('active');
    });

    // Image slider
    const slides = document.querySelector('.slides');
    const slide = document.querySelectorAll('.slide');
```

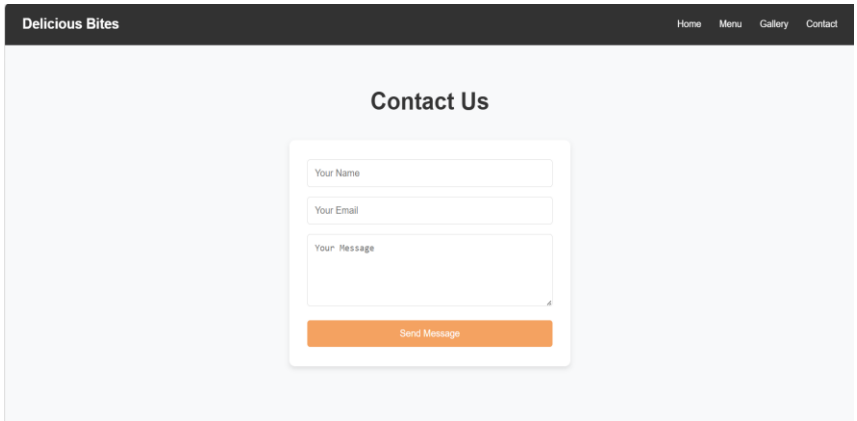
```html
    // Image slider
    const slides = document.querySelector('.slides');
    const slide = document.querySelectorAll('.slide');
    const prevBtn = document.querySelector('.prev');
    const nextBtn = document.querySelector('.next');
    let currentIndex = 0;
    const totalSlides = slide.length;
    function showSlide(index) {
        if (index < 0) {
            currentIndex = t const totalSlides: number
        } else if (index >= totalSlides) {
            currentIndex = 0;
        } else {
            currentIndex = index;
        }
        slides.style.transform = `translateX(-${currentIndex * 100}%)`;
    }
    prevBtn.addEventListener('click', () => {
        showSlide(currentIndex - 1);
    });
    nextBtn.addEventListener('click', () => {
        showSlide(currentIndex + 1);
    });
    // Auto slide (optional)
    setInterval(() => {
        showSlide(currentIndex + 1);
    }, 5000);
</script>
```

OUTPUT:


Delicious Bites
Home
Menu
Gallery
Contact
Welcome to Delicious Bites
Experience the finest cuisine in a cozy atmosphere
View Menu


Delicious Bites
Home
Menu
Gallery
Contact
Our Menu
Grilled Salmon
Grilled Salmon
Fresh Atlantic salmon grilled to perfection with herbs and lemon.
$24.99
Beef Steak
Juicy ribeye steak cooked to your liking with garlic butter.
$29.99
Pasta Carbonara
Pasta Carbonara
Classic Italian pasta with pancetta, eggs, and Parmesan cheese.
$18.99
Chocolate Cake
Rich and moist chocolate cake with vanilla frosting.
$8.99


Delicious Bites
Home
Menu
Gallery
Contact
Gallery



EXPLANATION:

This task creates a restaurant landing page using HTML, CSS, and JavaScript to display the restaurant's information in an attractive way. A navigation bar is added with links such as Home, Menu, Gallery, and Contact to help users easily move between sections. The menu section is designed using CSS cards to present food items with images, descriptions, and prices. A JavaScript image slider is implemented in the gallery section to display images with smooth transitions and better user experience.

Task 3: AI-Powered Event Registration Form

Scenario

SR University is hosting a technical fest and requires an online registration form

with real-time validation to ensure accurate user input.

Tasks

1. Ask Copilot to generate an HTML form with the following fields:

o Name

o Email

o Phone Number

o Department

o Event Selection

2. Use AI assistance to style the form using CSS for readability and

accessibility.

3. Implement JavaScript validation using Copilot:

o Email format validation

o Phone number length validation

o Required field checks

Expected Outcome

• User-friendly and visually appealing form

• Real-time validation messages

• Prevention of invalid submissions

PROMPT:

Generate a responsive Event Registration Form for a technical fest using a single HTML file that includes HTML, CSS, and JavaScript.

Requirements

1. HTML Form Structure

- Create a form with the following input fields:
    - Name (text field)
    - Email (email field)
    - Phone Number (number or text field)
    - Department (dropdown list with options like CSE, ECE, EEE, Mechanical, Civil)
    - Event Selection (dropdown or radio buttons with 3–4 event options)
- Add a Submit button.

2. CSS Styling

- Write CSS inside a <style> tag in the same HTML file.
- Center the form on the page with a clean layout.
- Use padding, borders, and spacing for readability.
- Add hover effects on the submit button.
- Ensure the form is responsive for mobile and desktop screens.

3. JavaScript Validation

- Write JavaScript inside a <script> tag at the bottom of the HTML file.
- Implement real-time validation for:
    - Email format validation using a regular expression.
    - Phone number validation (must be exactly 10 digits).
    - Required field validation (fields cannot be empty).
- Display clear error messages below the input fields.

4. User Experience

- Prevent form submission if validation fails.
- Show success message when all inputs are valid.

CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Event Registration Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f4f4f4;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
        }
        .container {
            background-color: white;
            padding: 20px;
            border-radius: 8px;
```

```
        .container {
            background-color: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
            max-width: 500px;
            width: 100%;
            box-sizing: border-box;
        }
        h1 {
            text-align: center;
            margin-bottom: 20px;
            color: #333;
        }
        .form-group {
            margin-bottom: 15px;
        }
        label {
```

```
        button {
            cursor: pointer;
            font-size: 16px;
            transition: background-color 0.3s;
        }
        button:hover {
            background-color: #0056b3;
        }
        .error {
            color: red;
            font-size: 14px;
            margin-top: 5px;
            display: block;
        }
        .success {
            color: green;
            font-size: 16px;
```

```html
</head>
<body>
    <div class="container">
        <h1>Event Registration Form</h1>
        <form id="registrationForm">
            <div class="form-group">
                <label for="name">Name:</label>
                <input type="text" id="name" name="name">
                <span class="error" id="nameError"></span>
            </div>
            <div class="form-group">
                <label for="email">Email:</label>
                <input type="email" id="email" name="email">
                <span class="error" id="emailError"></span>
            </div>
            <div class="form-group">
```

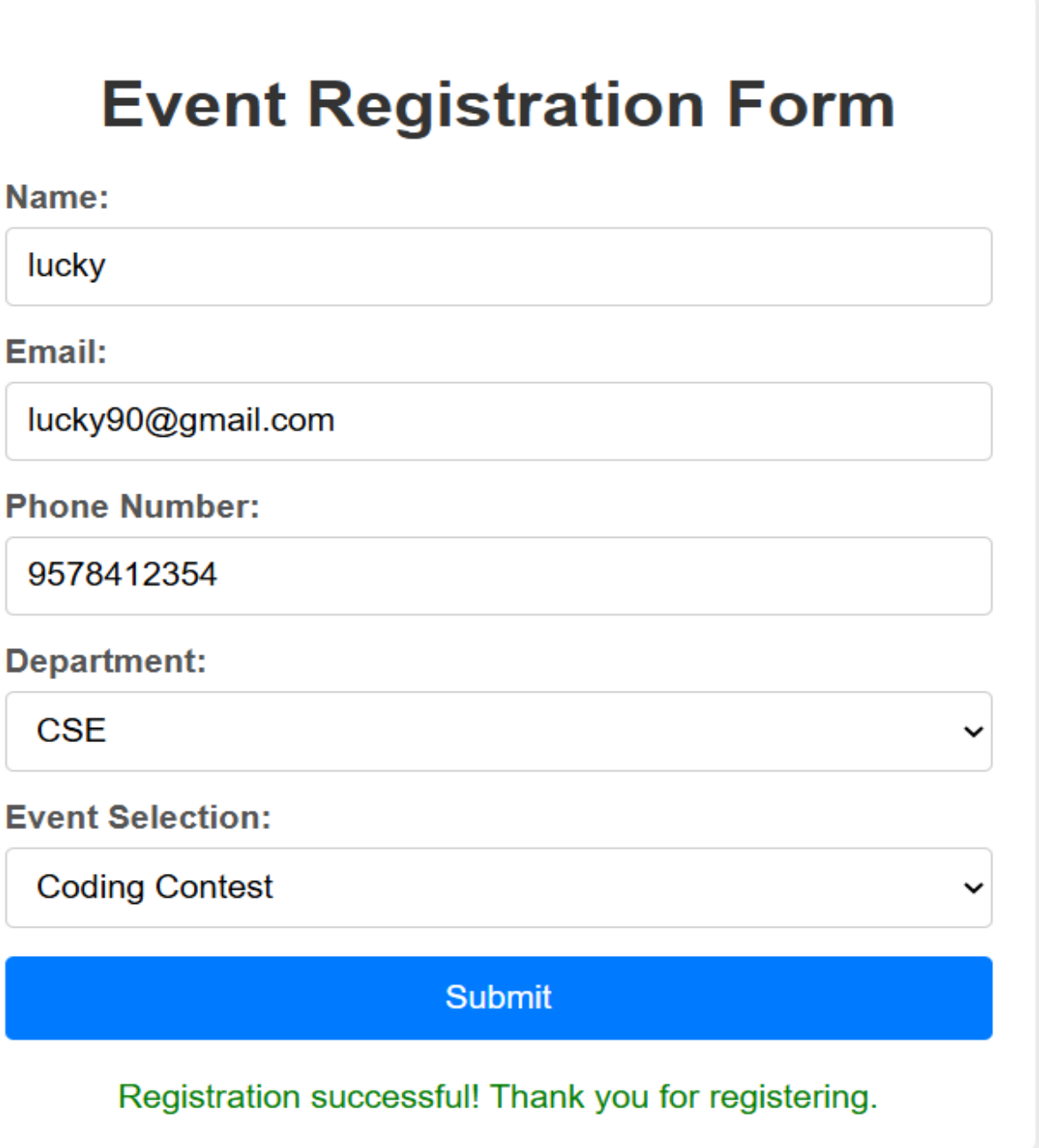
```html
            <div class="form-group">
            </div>
            <div class="form-group">
                <label for="department">Department:</label>
                <select id="department" name="department">
                    <option value="">Select Department</option>
                    <option value="CSE">CSE</option>
                    <option value="ECE">ECE</option>
                    <option value="EEE">EEE</option>
                    <option value="Mechanical">Mechanical</option>
                    <option value="Civil">Civil</option>
                </select>
                <span class="error" id="departmentError"></span>
            </div>
            <div class="form-group">
                <label for="event">Event Selection:</label>
                <select id="event" name="event">
                    <option value="">Select Event</option>
                    <option value="Hackathon">Hackathon</option>
                    <option value="Coding Contest">Coding Contest</option>
                    <option value="Robotics Workshop">Robotics Workshop</option>
```

```
    // Real-time validation
    document.getElementById('name').addEventListener('input', validateName);
    document.getElementById('email').addEventListener('input', validateEmail);
    document.getElementById('phone').addEventListener('input', validatePhone);
    document.getElementById('department').addEventListener('change', validateDepartment);
    document.getElementById('event').addEventListener('change', validateEvent);

    form.addEventListener('submit', function(event) {
        event.preventDefault();
        const isValid = validateForm();
        if (isValid) {
            successMessage.style.display = 'block';
            // Optionally, you can reset the form here: form.reset();
        } else {
            successMessage.style.display = 'none';
        }
    });

    function validateForm() {
        const nameValid = validateName();
        const emailValid = validatePhone();
        const phoneValid = validatePhone();
```

o

```
        function validateDepartment() {
            const department = document.getElementById('department').value;
            const error = document.getElementById('departmentError');
            if (department === '') {
                error.textContent = 'Please select a department.';
                return false;
            } else {
                error.textContent = '';
                return true;
            }
        }

        function validateEvent() {
            const event = document.getElementById('event').value;
            const error = document.getElementById('eventError');
            if (event === '') {
                error.textContent = 'Please select an event.';
                return false;
            } else {
                error.textContent = '';
                return true;
            }
        }
    </script>
</body>
</html>
```

OUTPUT:



EXPLANATION:

This task develops an AI-powered event registration form for a technical fest using HTML, CSS, and JavaScript. The form allows participants to enter important details such as name, email address, phone number, department, and selected event. CSS styling is applied to make the form visually appealing, organized, and easy to use on different screen sizes. JavaScript validation is implemented to ensure correct user input by checking the email format, validating the phone number length, and preventing form submission if required fields are empty.

Task 4: AI-Assisted E-Commerce Product Page

Scenario

A startup is developing a basic e-commerce product page to showcase products

and manage simple cart functionality.

Tasks

1. Use Copilot to generate a grid-based product catalog using HTML and

CSS.

2. Implement an “Add to Cart” feature using JavaScript.

3. Modify AI-generated code to include:

o Cart item counter at the top-right corner

o Visual feedback when a product is added

Expected Outcome

• Functional product catalog

• Working cart counter

• Interactive user experience

PROMPT:

Generate a basic E-Commerce Product Page using a single HTML file with HTML, CSS, and JavaScript.

Requirements

1. HTML Structure

- Create a product catalog displaying at least 6 products.
- Each product should include:
    - Product Image
    - Product Name
    - Short Description
    - Price
    - Add to Cart button.

2. Product Layout

- Use CSS Grid or Flexbox to design a grid-based product catalog.
- Ensure products are displayed in responsive cards.

3. Cart Feature

- Implement an “Add to Cart” functionality using JavaScript.
- When the user clicks the Add to Cart button, the item should be added to the cart.

4. Cart Counter

- Add a cart icon with an item counter at the top-right corner of the page.
- The counter should increase automatically whenever a product is added.

5. Visual Feedback

- Show visual feedback when a product is added to the cart.
    - Example: change button text to “Added” or show a small notification message.
    - Add smooth animations or transitions.

6. Styling

- Write CSS inside a <style> tag.
- Use modern UI design with shadows, hover effects, and spacing.
- Make the page responsive for mobile and desktop screens.

7. JavaScript

- Write JavaScript inside a <script> tag at the bottom of the HTML file.
- Handle the cart counter update and visual feedback.

CODE:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>E-Commerce Product Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f8f9fa;
            color: #333;
        }
        header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            padding: 20px;
            background-color: #fff;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
```

```
.products {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
    gap: 20px;
    padding: 20px;
    max-width: 1200px;
    margin: 0 auto;
}
.product-card {
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    overflow: hidden;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}
.product-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 4px 16px rgba(0,0,0,0.2);
}
.product-card img {
    width: 100%;
    height: 150px;
```

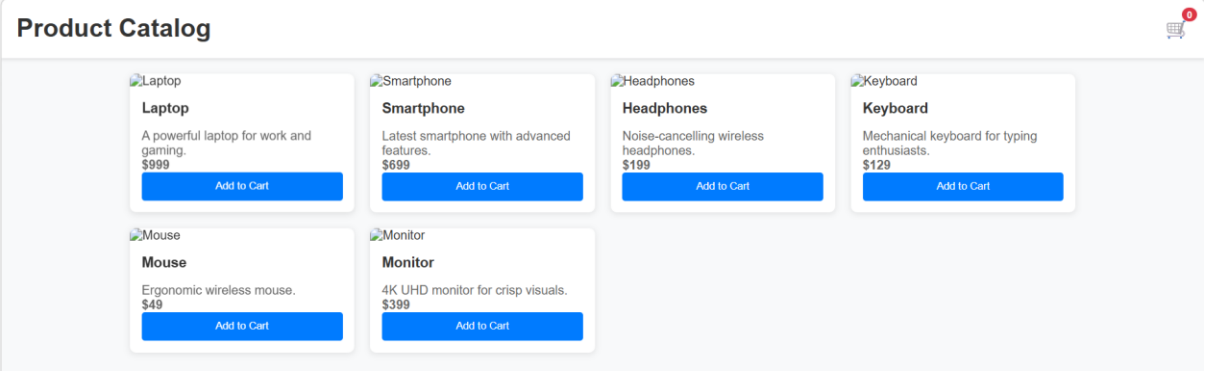
```
.add-to-cart:hover {
    background-color: #0056b3;
}
.add-to-cart.added {
    background-color: #28a745;
    cursor: default;
}
.notification {
    position: fixed;
    bottom: 20px;
    right: 20px;
    background-color: #28a745;
    color: white;
    padding: 10px 20px;
    border-radius: 4px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    opacity: 0;
    transform: translateY(20px);
    transition: opacity 0.3s ease, transform 0.3s ease;
    z-index: 1000;
}
```

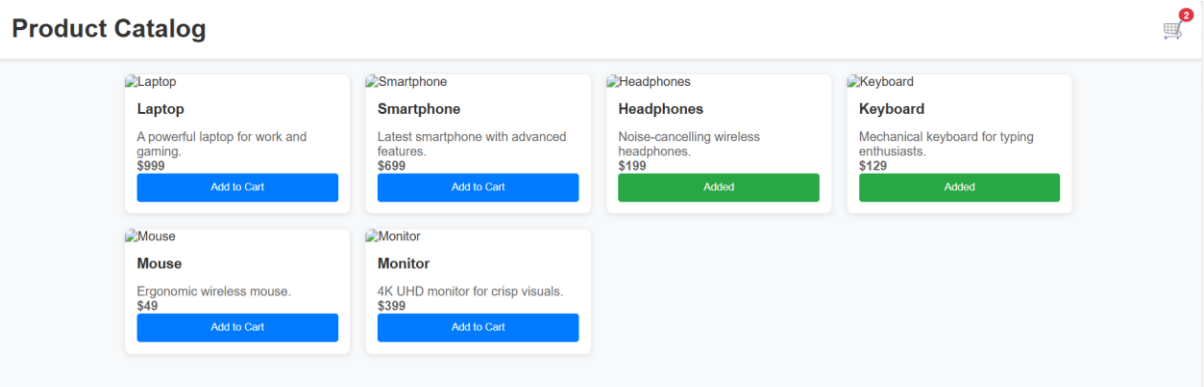
```html
</head>
<body>
    <header>
        <h1>Product Catalog</h1>
        <div class="cart">
            <span class="cart-icon">🛒</span>
            <span class="cart-count" id="cartCount">0</span>
        </div>
    </header>
    <main class="products">
        <div class="product-card">
            <img src="
            <h3>Laptop</h3>
            <p>A powerful laptop for work and gaming.</p>
            <p class="price">$999</p>
            <button class="add-to-cart" data-product="Laptop">Add to Cart</button>
        </div>
        <div class="product-card">
            <img src="
            <h3>Smartphone</h3>
            <p>Latest smartphone with advanced features.</p>
            <p class="price">$699</p>
            <button class="add-to-cart" data-product="Smartphone">Add to Cart</button>
```

```html
        <div class="product-card">
            <img src="
            <h3>Mouse</h3>
            <p>Ergonomic wireless mouse.</p>
            <p class="price">$49</p>
            <button class="add-to-cart" data-product="Mouse">Add to Cart</button>
        </div>
        <div class="product-card">
            <img src="
            <h3>Monitor</h3>
            <p>4K UHD monitor for crisp visuals.</p>
            <p class="price">$399</p>
            <button class="add-to-cart" data-product="Monitor">Add to Cart</button>
        </div>
    </main>
    <div class="notification" id="notification">Product added to cart!</div>
    <script>
        let cartCount = 0;
        const cartCountElement = document.getElementById('cartCount');
        const notification = document.getElementById('notification');
```

```html
            <button class="add-to-cart" data-product="Monitor">Add to Cart</butto
        </div>
    </main>
    <div class="notification" id="notification">Product added to cart!</div>
    <script>
        let cartCount = 0;
        const cartCountElement = document.getElementById('cartCount');
        const notification = document.getElementById('notification');
        const addToCartButtons = document.querySelectorAll('.add-to-cart');
        addToCartButtons.forEach(button => {
            button.addEventListener('click', function() {
                const productName = this.getAttribute('data-product');
                cartCount++;
                cartCountElement.textContent = cartCount;
                this.textContent = 'Added';
                this.classList.add('added');
                this.disabled = true;
                // Show notification
                notification.classList.add('show');
                setTimeout(() => {
                    notification.classList.remove('show');
                }, 2000);
            });
        });
    </script>
</body>
```

OUTPUT:





EXPLANATION:

This task develops a basic e-commerce product page using HTML, CSS, and JavaScript to display products in an organized way. The products are arranged in a grid-based layout using CSS, where each product card contains an image, product name, description, price, and an “Add to Cart” button. JavaScript is used to implement the cart functionality, which increases the cart counter whenever a user adds a product. Additionally, visual feedback is provided by temporarily changing the button text and colour when a product is added, creating a more interactive and user-friendly shopping experience.

Task 5: Create a Responsive Web Page Layout

Instructions:

• Design a basic web page layout with a header, main content area, and

footer using HTML and CSS

Use AI to assist in generating responsive CSS for different screen sizes.

• Ensure the layout is clean and visually organized.

Expected Output:

• A responsive web page with:

• Header with navigation links

• Main section with placeholder text/images

• Footer with copyright or contact info

• Layout adapts correctly to desktop, tablet, and mobile screen sizes.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots. Use AI to assist in generating responsive CSS for different screen sizes.

• Ensure the layout is clean and visually organized.

Expected Output:

• A responsive web page with:

• Header with navigation links

• Main section with placeholder text/images

• Footer with copyright or contact info

• Layout adapts correctly to desktop, tablet, and mobile screen sizes.

PROMPT:

Generate a responsive web page layout using HTML and CSS in a single HTML file.

Requirements

1. HTML Structure

- Use semantic HTML5 elements.
- Create the following sections:
    - Header with a website title and navigation links (Home, About, Services, Contact).
    - Main Content Section with placeholder text and one or two placeholder images.
    - Footer with copyright information or contact details.

2. Navigation

- Add a navigation menu inside the header.
- The navigation links should be horizontally aligned on desktop screens.

3. Responsive Design

- Write CSS inside a <style> tag in the same HTML file.
- Use Flexbox or CSS Grid to structure the layout.
- Implement media queries for different screen sizes:
    - Desktop
    - Tablet
    - Mobile
- On smaller screens, the navigation menu should stack vertically.

4. Styling

- Use a clean and modern layout with proper spacing, padding, and colors.
- Add basic hover effects for navigation links.
- Ensure the page is visually organized and easy to read

CODE:

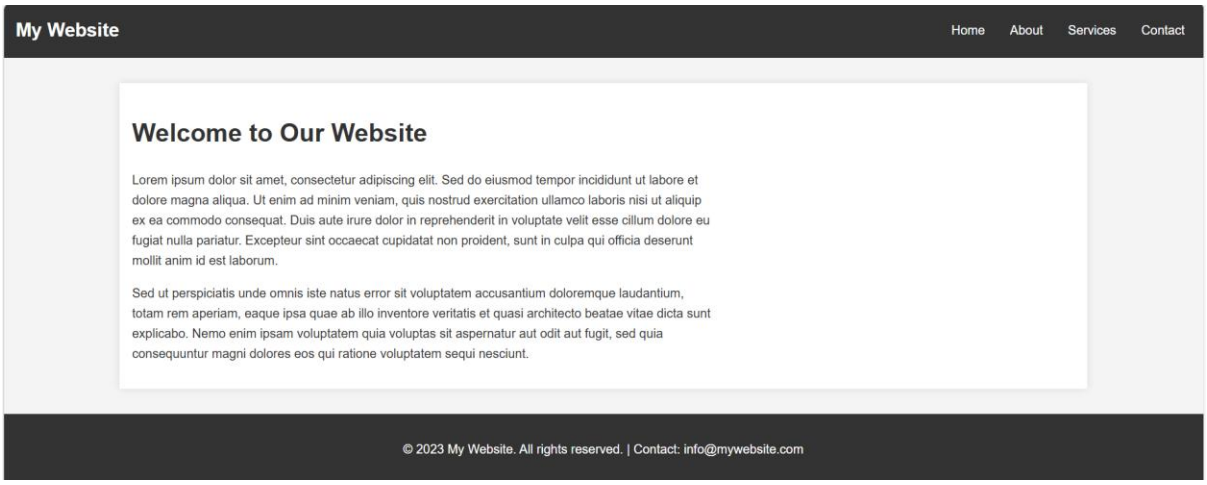
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Web Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            line-height: 1.6;
            color: #333;
            background-color: #f4f4f4;
        }
        header {
            background-color: #333;
            color: white;
            padding: 1rem;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }
        .logo {
            font-size: 1.5rem;
```

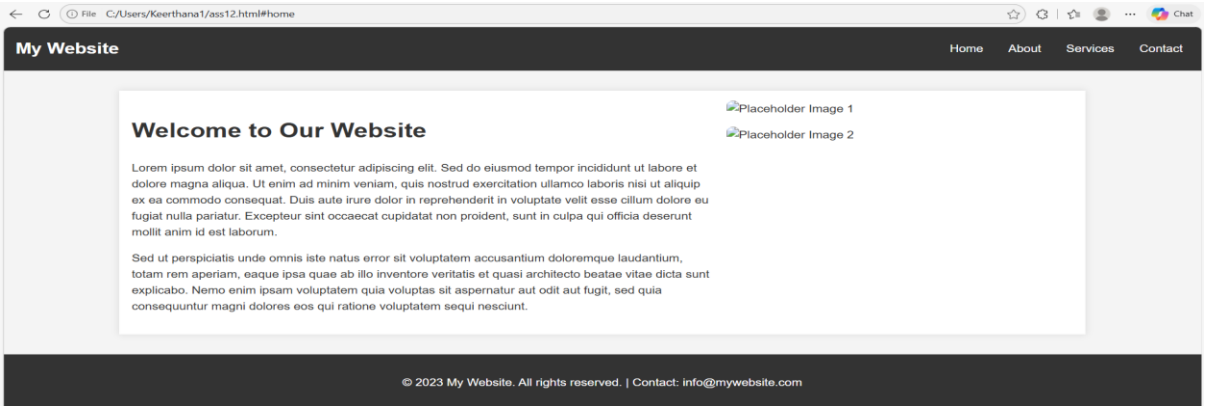
```
        }
        nav a:hover {
            background-color: #555;
            border-radius: 4px;
        }
        main {
            max-width: 1200px;
            margin: 2rem auto;
            padding: 1rem;
            background-color: white;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        .content {
            display: flex;
            flex-wrap: wrap;
            gap: 1rem;
        }
        .text {
            flex: 1 1 60%;
            min-width: 300px;
        }
```

```html
            </div>
            <div class="images">
                <img src="https://via.placeholder.com/400x300/cccccc/000000?text=Placeholder+Image+1" alt="Placeho
                <img src="https://via.placeholder.com/400x300/cccccc/000000?text=Placeholder+Image+2" alt="Placeho
            </div>
        </section>
    </main>
    <footer>
        <p>&copy; 2023 My Website. All rights reserved. | Contact: info@mywebsite.com</p>
    </footer>
</body>
</html>
```

```html
</head>
<body>
    <header>
        <div class="logo">My Website</div>
        <nav>
            <ul>
                <li><a href="#home">Home</a></li>
                <li><a href="#about">About</a></li>
                <li><a href="#services">Services</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <section class="content">
            <div class="text">
                <h1>Welcome to Our Website</h1>
                <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut la
                <p>Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantiu
            </div>
            <div class="images">
```

OUTPUT:





EXPLANTION:

This task creates a responsive web page using HTML and CSS. The page contains a header with navigation links, a main section with sample text or images, and a footer with copyright or contact information. CSS is used to style the layout and make the page look clean and organized. Media queries are used to make the webpage adjust properly for desktop, tablet, and mobile screens.